Labs 7-8

This week covers topics from

- Missing and Outliers
- Naive Bayes
- Clustering
- Silhouettes

Submission And Assignment Instructions

Submit your `Lab7-8_lastname_firstname.pdf` file on canvas with your answers clearly marked (✅) and code commented. Skeleton code and/or comments are provided throughout to help you.

Formatting Notes:

- ❓ Questions you must answer or tasks are marked with a "❓ "
- ✅ Answers you should give are marked with "✅ Answer:"
- *Helpful hints are usually given in italics*
- Code you need to write is marked with `## ❓ YOUR CODE HERE`
  - ⭐ Bonus questions are marked with a "⭐ "
  - 🍓 Bonus answers are marked with a "🍓 "

Please don't erase ANY of these markings! They're to help the TAs find your answers and code as much as they're to help you find the questions.

Section 1 - Bayes Rule

First we'll do some basic math with Bayes Rule:

$$P(A|B) = \frac{P(B|A)\ P(A)}{P(B)} \tag{1}$$

A medical research team is developing a new test for a rare genetic condition. They have gathered the following data:

- The prevalence of the genetic condition in the general population is 1% (P(C) = 0.01).
- The test has a sensitivity of 95%, meaning it correctly identifies 95% of people who have the condition (P(T|C) = 0.95).
- The test has a specificity of 90%, meaning it correctly identifies 90% of people who don't have the condition (P(T'|C') = 0.90).

> *The ' is sometimes used to denote the complement of a set in probability. P(T) is the probability of a positive test, so P(T') is the probability of a negative test. P(T) = 1 - P(T'). This also applies to conditional probabilities! P(T|C) = 1 - P(T'|C)*

- *Hint: Keep in mind that P(T) can be calculated using the law of total probability:* $P(T) = P(T|C)P(C) + P(T|C')P(C')$

❓ Using Bayes' rule, calculate the probability that a person actually has the genetic condition given that they tested positive (P(C|T)).

Show your work and express your final answer as a probability rounded to two decimal places.

✅ Answer:

- P(T | no condition) = 1 – 0.90 = 0.10
- P(T) = (0.95)(0.01) + (0.10)(0.99) = 0.0095 + 0.099 = 0.1085
- P(C | T) = (0.95)(0.01) / 0.1085 = 0.0095 / 0.1085 = 0.09

Section 2 - Real data

We've seen clusters or groups of data many times in class, usually clusters from externally generated labels. Now we're going to use unsupervised clustering to explore the structure of our data.

2.1: Load the dataset

This dataset is from a paper investigating subtypes of T-lineage acute lymphoblastic leukemia.

❓ Load the truncated provided datafile (`data_09_TALL_tpm_reduced.csv`).

Some checks to help you:

- This dataset is TPM (transcripts per million) gene expression data from 1,335 different tumor samples across 300 genes. Check that your dataset is the expected size after loading it.
- The first column of your data should be long strings (e.g. `ENSG00000004939.15`). These are ENSEMBL IDs, identification strings for particular gene isoforms. *Load these as the index column of your dataframe.*

In [13]:
```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# load local file "data08.csv"
#from google.colab import files

#uploaded = files.upload()

#for fn in uploaded.keys():
  #print('User uploaded file "{name}" with length {length} bytes'.format(
      #name=fn, length=len(uploaded[fn])))

## ❓ YOUR CODE HERE
df = pd.read_csv("data_09_TALL_tpm_reduced.csv", index_col=0)
print(df.shape)
display(df.head())
print(df.index[:5])
```

```
(300, 1335)
```

X	PAUCKE	PARLHD	PARMJA	PARVRF	PARYMD	PASXLM	PATVDU	PATZXK	PAUCIZ	PAUCVB	...	PAUAJA	PA
ENSG00000000971.15	0.03	0.11	0.01	4.73	6.61	0.12	3.57	0.05	0.00	3.12	...	0.03	
ENSG00000001626.15	0.00	0.00	0.20	0.00	0.16	0.00	0.00	0.00	0.03	0.00	...	0.01	
ENSG00000003436.16	3.77	3.75	9.16	2.89	11.02	23.81	3.63	0.24	4.70	3.78	...	6.39	
ENSG00000004939.15	0.39	6.40	31.64	2.24	45.40	0.12	5.54	13.63	2.24	33.52	...	0.60	
ENSG00000005381.7	0.30	6.47	1.22	0.81	0.73	7.81	18.78	22.97	0.83	80.46	...	8.30	

5 rows × 1335 columns

```
Index(['ENSG00000000971.15', 'ENSG00000001626.15', 'ENSG00000003436.16',
       'ENSG00000004939.15', 'ENSG00000005381.7'],
      dtype='str', name='X')
```

❓ Once again, we've loaded gene expression data with genes as rows and samples as columns. Transpose the data to make the genes (features) into columns and the column names into sample IDs.

In [14]:
```
# YOUR CODE HERE
dfTrans = df.T
print(dfTrans.shape)
dfTrans.head()
```

```
(1335, 300)
```

Out[14]:

X	ENSG00000000971.15	ENSG00000001626.15	ENSG00000003436.16	ENSG00000004939.15	ENSG00000005381.7	ENSG00
PAUCKE	0.03	0.00	3.77	0.39	0.30	
PARLHD	0.11	0.00	3.75	6.40	6.47	
PARMJA	0.01	0.20	9.16	31.64	1.22	
PARVRF	4.73	0.00	2.89	2.24	0.81	
PARYMD	6.61	0.16	11.02	45.40	0.73	

5 rows × 300 columns

2.2: Hierarchical Clustering

We're going to use hierarchical clustering to visualize these data and create clusters. Since this is a large dataset, we're going to compute the linkage matrix separately using `fastcluster` before visualizing with seaborn.

❓ 2.2 Use `fastcluster.linkage` to compute the hierarchical clustering with complete linkage. This output is called a stepwise dendrogram. Print the first several (look at 15-20) rows of the stepwise dendrogram numpy array.

In [15]:
```
#%%capture
#!pip install fastcluster
import fastcluster
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster

## YOUR CODE HERE
X = dfTrans.values
y = fastcluster.linkage(X,
                        method="complete",
                        metric="euclidean")
print(y.shape)
print(y[:20])
```

```
(1334, 4)
[[ 335.          925.           49.86833163    2.        ]
 [   3.         1335.           70.12810635    3.        ]
 [ 900.          976.           86.18298556    2.        ]
 [ 350.          849.           92.43486518    2.        ]
 [  66.          267.           93.87894865    2.        ]
 [1015.         1043.           97.99988418    2.        ]
 [ 269.         1336.          104.17701474    4.        ]
 [ 456.         1337.          110.70595241    3.        ]
 [ 461.          893.          112.19822191    2.        ]
 [1032.         1343.          131.14502316    3.        ]
 [ 399.          938.          137.59028381    2.        ]
 [1342.         1344.          141.556766      6.        ]
 [  30.           47.          142.39807197    2.        ]
 [ 626.         1262.          142.594313      2.        ]
 [ 553.          714.          143.38061619    2.        ]
 [ 658.          685.          143.83076514    2.        ]
 [ 487.         1339.          144.50486843    3.        ]
 [ 964.         1266.          149.89764508    2.        ]
 [ 660.         1123.          151.25511925    2.        ]
 [1248.         1345.          151.98152256    3.        ]]
```

A couple questions about this output:

- ❓ 2.2.1 What does each row of this output represent? (1 sentence)
  - ✅ Answer: Each row is one merge (two clusters or single samples) joined into a new cluster.

- ❓ 2.2.2 What does each column of this output represent? *There should be 4 columns.*
  - ✅ Answer: Columns 0 and 1 are the indices of the two clusters merged. An index of 1335 or higher means a cluster formed at an earlier step. Column 2 is the complete-linkage (Euclidean) distance between them and column 3 is the number of samples in the new cluster.

- ❓ 2.2.3 How many rows does this output have? How are the rows of data arranged? (1 sentence)
  - ✅ Answer: There are 1334 rows and n – 1 merges turn 1335 samples into one cluster. The rows are ordered from smallest to largest merge distance.

- ❓ 2.2.4 By this linkage metric, what are the indices of the two most similar samples? *These are tumor sample indices, not gene indices, since we're computing the clustering on rows of our dataframe. One way to know that is that you have only 300 genes, and both these indices are >300!*
  - ✅ Answer: Samples 335 and 925 merge first at distance 49.87.

- ❓ 2.2.5 The first cluster formed has index 1335. At what distance does this cluster recombine with another cluster? *Hint: You should not have to print out more rows of this output or write more code to answer this question.*
  - ✅ Answer: At 70.13; row 2 merges cluster 1335 with sample 3.

2.3: Visualizing Clustering

❓ 2.3 Use `sns.clustermap` to visualize these data into a complete linkage heatmap. *This question has many steps, please read carefully.*

Make sure to:

- use the `fcluster` function to divide your clustered rows (samples) into 15 clusters
  - `fcluster` lets you divide a dendrogram into clusters multiple ways. Rather than choosing a height on the dendrogram, choose a maximum number of clusters.
- use the `row.colors` argument in `sns.clustermap` to visually divide the rows (samples) into those 15 distinct clusters.
- provide the row linkage clustering you already computed using the `row_linkage` argument of the `sns.clustermap` function.
- include a title to identify the type of clustering you used and other relevant information (clustering type, linkage type, distance metric)
- label your colorbar (what is this scale?)
- use `standard_scale = 1` to map the expression of each gene onto a scale between 0 and 1 (hint: this changes what the colorbar scale is). *Does this change how the row linkage is produced?*
- use `method = "complete"` to use complete linkage clustering on the genes/columns (in addition to the rows, which you have already clustered).
- label the x and y-axes as either "Genes" or "Samples", but remove individual sample or ENSEMBL IDs for brevity
- include the clustering dendrograms for both genes and samples in your final heatmap

In [ ]:
```
#%%capture
#!pip install fastcluster
import fastcluster
from scipy.cluster.hierarchy import dendrogram, fcluster

## ❓ YOUR CODE HERE
num_clusters = 15

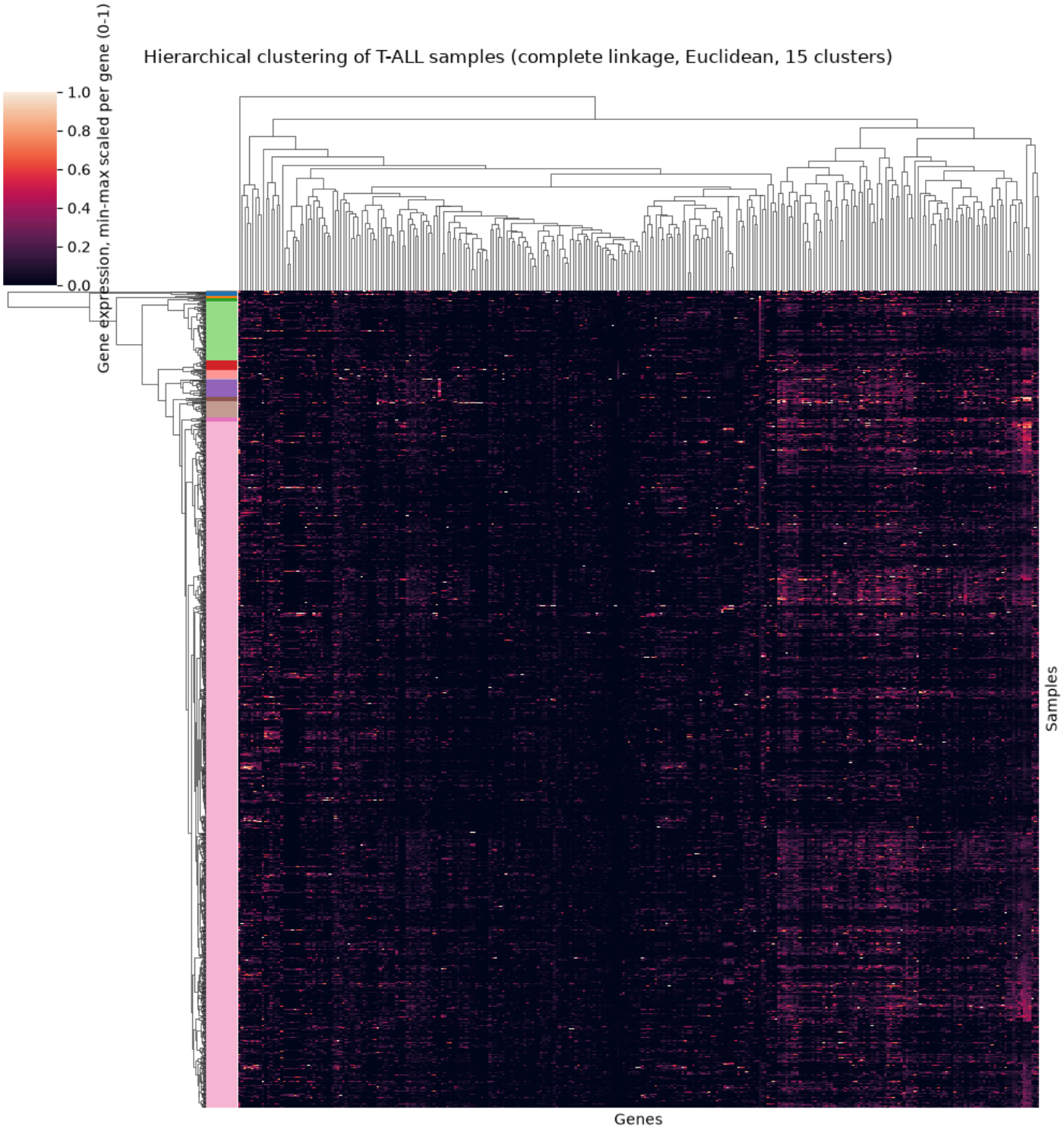
# cut the dendrogram to create clusters
row_clusters = fcluster(y, t=num_clusters, criterion ="maxclust")

# Example: Create a color palette for the clusters
palette = sns.color_palette("tab20", num_clusters)
cluster_colors = [palette[i-1] for i in row_clusters]

# Create a clustermap with colored row labels based on clusters
g = sns.clustermap(
    dfTrans,
    row_linkage=y,
    method="complete",
    metric="euclidean",
    standard_scale=1,
    row_colors=cluster_colors,
    xticklabels=False,
    yticklabels=False,
    figsize=(10, 10),
    cbar_kws={"label": "Gene expression, min-max scaled per gene (0-1)"},
)

# make sure to label the colorbar

# add axis labels and title
g.ax_heatmap.set_xlabel("Genes")
g.ax_heatmap.set_ylabel("Samples")
g.fig.suptitle("Hierarchical clustering of T-ALL samples (complete linkage, Euclidean, 15 clusters)", y=1.05)
plt.show()
```

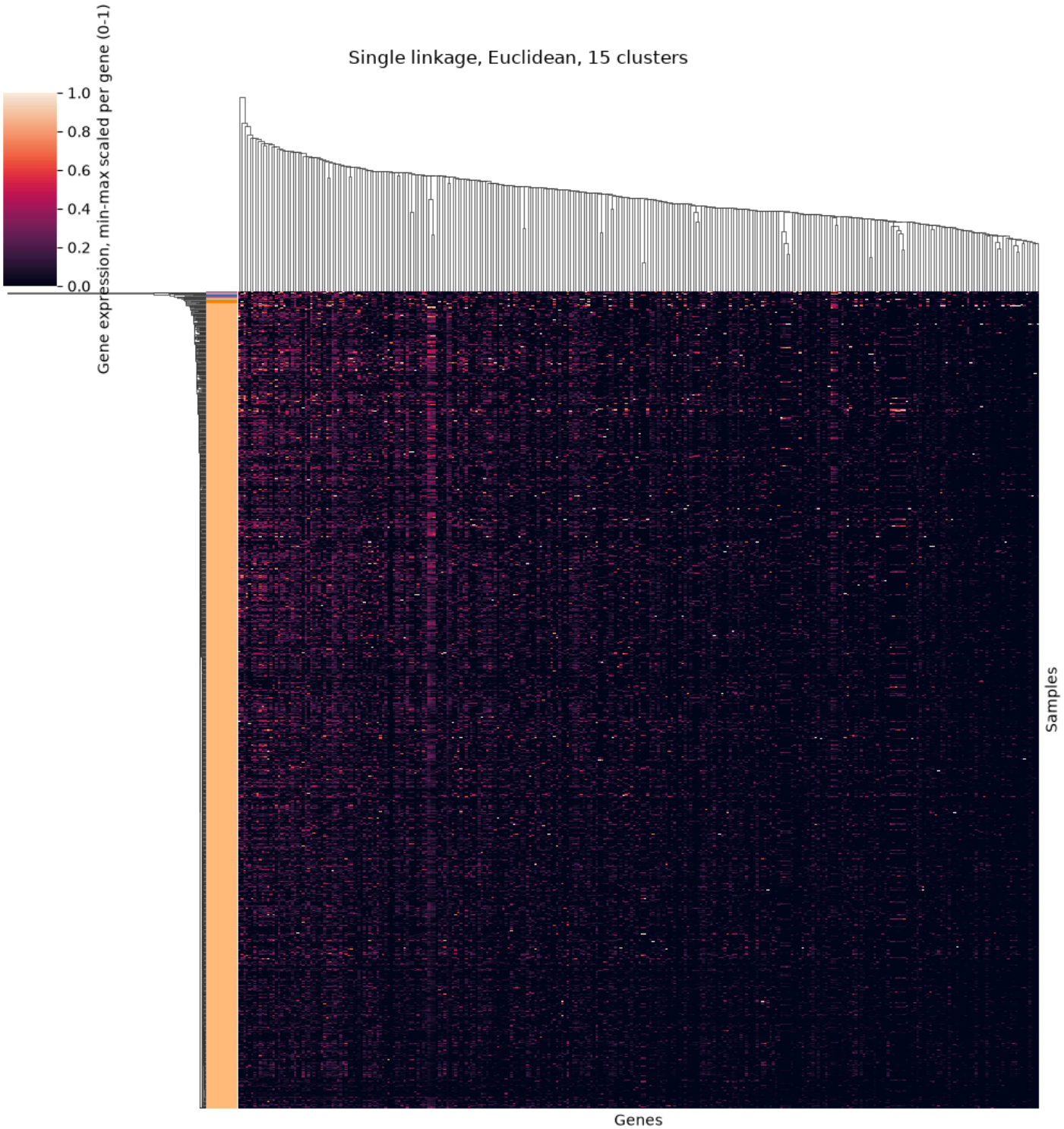


2.4 Comparing Types of Clustering

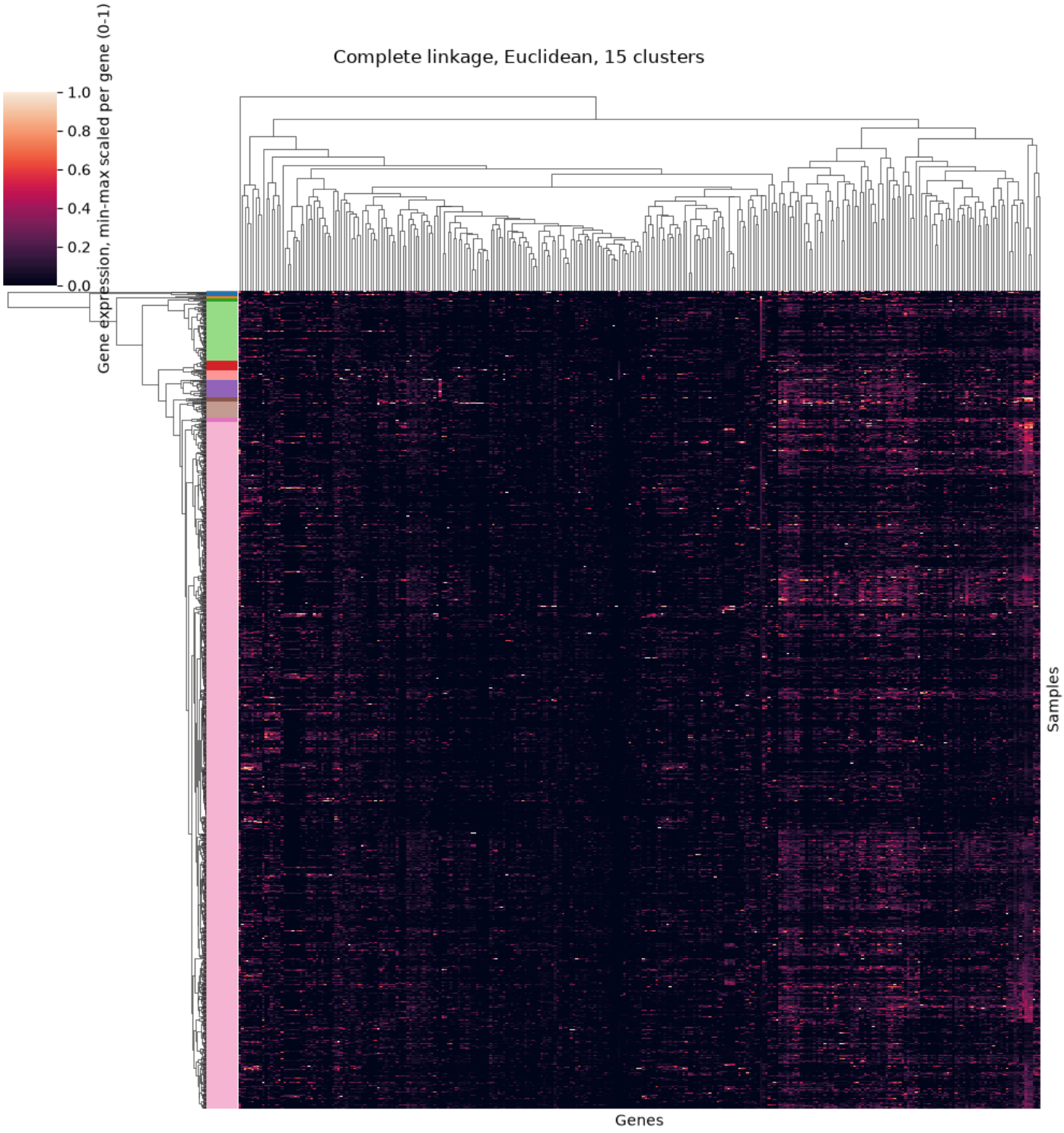
❓ 2.4.1 Compare single-linkage, complete-linkage, and the average linkage on this data, plotting each heatmap as before (you should need 2 additional heatmaps). Which sort of linkage gives the most uneven clusters by size (number of samples per cluster)? Based on what you know about how that linkage is calculated, why is that the case?

In [ ]:
```
## YOUR CODE HERE
# Claude was used to debug and create efficient for loop
linkage_types=['single', 'complete', 'average']
for linkage_type in linkage_types:
    Z = fastcluster.linkage(dfTrans,
                            method=linkage_type,
                            metric="euclidean")
    row_clusters = fcluster(Z, t=15, criterion="maxclust")
    palette = sns.color_palette("tab20", 15)
    row_colors = [palette[i-1] for i in row_clusters]
    g = sns.clustermap(dfTrans,
                       row_linkage=Z,
                       method=linkage_type,
```

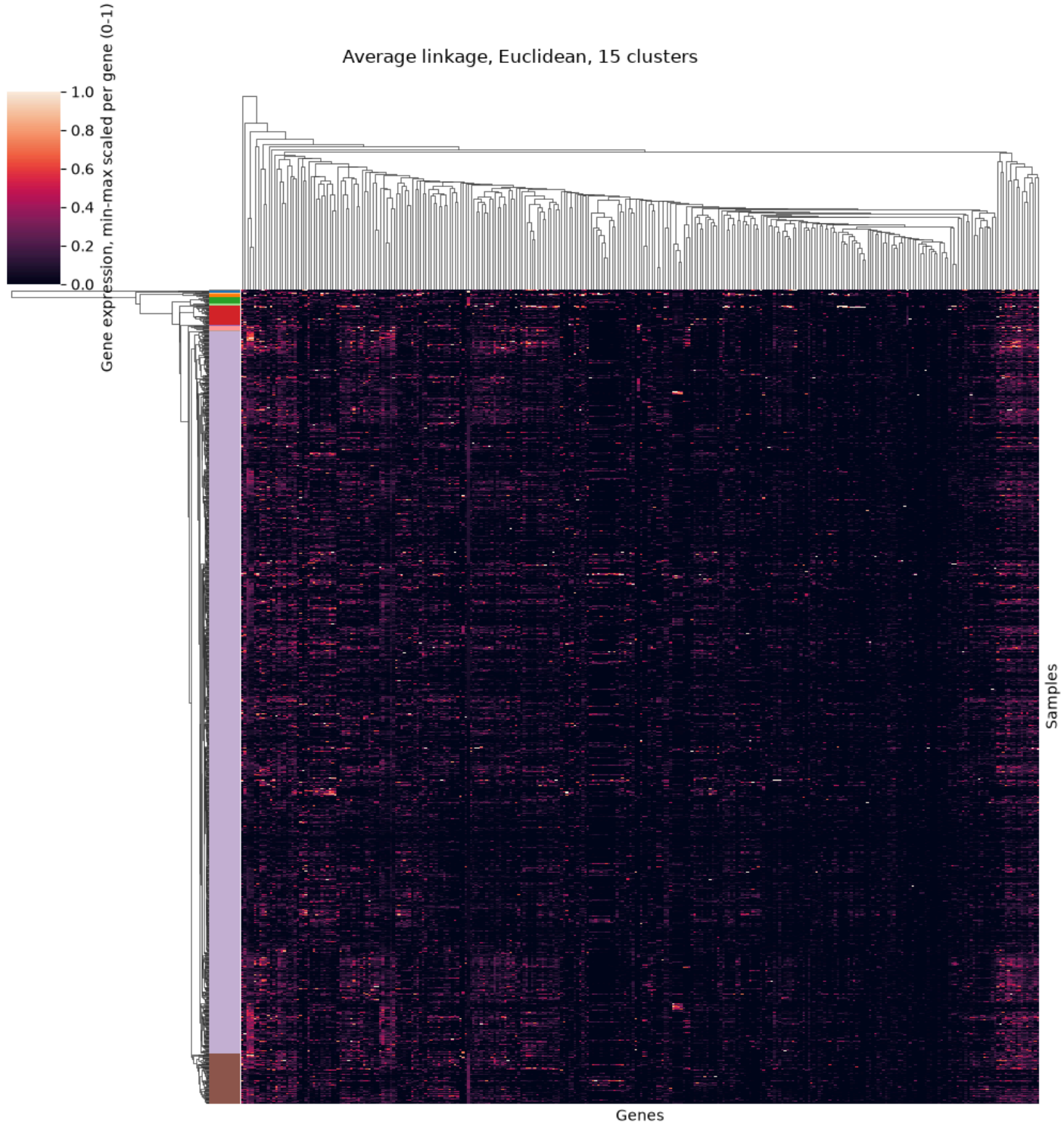
```
                      metric="euclidean",
                      standard_scale=1,
                      row_colors=row_colors,
                      xticklabels=False,
                      yticklabels=False,
                      figsize=(10, 10),
                      cbar_kws={"label": "Gene expression, min-max scaled per gene (0-1)"})
    g.ax_heatmap.set_xlabel("Genes")
    g.ax_heatmap.set_ylabel("Samples")
    g.fig.suptitle(f"{linkage_type.capitalize()} linkage, Euclidean, 15 clusters", y=1.05)
    plt.show()
    print(linkage_type, "cluster sizes:", sorted(np.bincount(row_clusters)[1:], reverse=True))
```




```
single cluster sizes: [np.int64(1316), np.int64(5), np.int64(2), np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.
int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(1)]
```



complete cluster sizes: [np.int64(1122), np.int64(97), np.int64(29), np.int64(26), np.int64(15), np.int64(15), np.int64(7), np.int64(6), np.int64(6), np.int64(5), np.int64(3), np.int64(1), np.int64(1), np.int64(1), np.int64(1)]




```
average cluster sizes: [np.int64(1186), np.int64(81), np.int64(31), np.int64(10), np.int64(8), np.int64(6), np.int64(3),
np.int64(3), np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(1)]
```

✅ Answer: Single linkage gives the most uneven clusters (1316 samples in one cluster, the other 14 clusters have less than 10 in each), compared with complete (largest 1122) and average (largest 1186). Single linkage merges clusters based on their closest pair of points, so a single cluster keeps growing by chaining through nearest neighbors, leaving outliers as singletons.

❓ 2.4.2 Are these hierarchical clustering methods sensitive to the scale of the data? Write some code to illustrate your answer.

In [18]:
```
## YOUR CODE HERE
from sklearn. preprocessing import StandardScaler
from collections import Counter
Z = fastcluster.linkage(dfTrans.values,
                        method="complete",
                        metric="euclidean")
clusters = fcluster(Z, t=15, criterion="maxclust")
scaler = StandardScaler()
ZScaled = fastcluster.linkage(scaler.fit_transform(dfTrans), method="complete")
clustersScaled = fcluster(ZScaled,
                          t=15,
```

```
                          criterion="maxclust")
print("Raw cluster sizes=", sorted(Counter(clusters).values()))
print("Scaled cluster sizes=", sorted(Counter(clustersScaled).values()))
```

```
Raw cluster sizes= [1, 1, 1, 1, 3, 5, 6, 6, 7, 15, 15, 26, 29, 97, 1122]
Scaled cluster sizes= [1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 3, 4, 4, 8, 1305]
```

✅ Answer (1 sentence): Yes, euclidean distance is dominated by high-expression genes, and z-scoring the genes changed the cluster sizes.

❓ 2.4.3 For each of these hierarchical methods, given the data that you have visualized, do you think choosing n = 15 clusters makes the most sense, based on the dendrograms? Why or why not?

✅ Answer: No, for all three linkages, a cut at 15 gives one giant cluster plus singletons and tiny clusters, not 15 meaningful groups. The biggest vertical gaps in the dendrograms are near the top and support only a few clusters. Also, the extreme raw TPM values suggest log-transforming the data before clustering.

2.5 Splits

❓ We didn't use a train-test split to try to evaluate any of these clustering techniques! Should we have used a train-test split? Why or why not? *Hint: Were these supervised or unsupervised techniques?*

✅ Answer(1 sentence): Clustering is unsupervised, there aren't labels to predict, so there's no test error for a held-out set to estimate.

Section 3 - Quantitative Comparison on Toy data

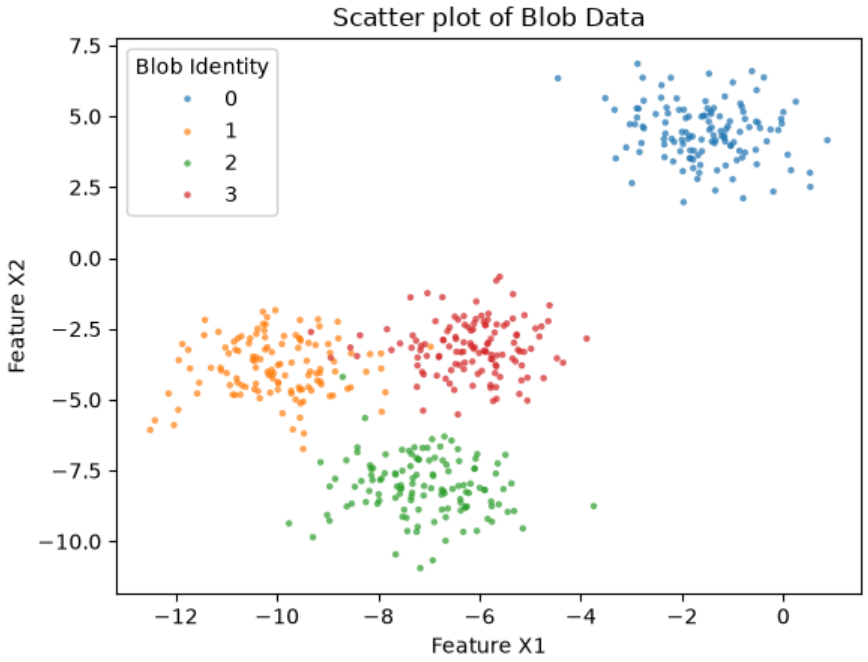
We've been arbitrarily dividing our tumor samples into 15 clusters, which doesn't seem to be a good fit for the dendrograms. We're going to take a quick break from the real data to use some toy data to understand k-means clustering and some quantitative measures that can help us evaluate different cluster formations.

Run the code below to create and visualize 4 clusters of toy data.

In [ ]:
```
from sklearn.datasets import make_blobs

# generate toy data
X, y = make_blobs(
    n_samples=500,
    n_features=2,
    centers=4,
    cluster_std=1,
    center_box=(-10.0, 10.0),
    shuffle=True,
    random_state=1)  # For reproducibility
X[0:5]
# This is only 2-D data, so we can directly visualize it!
# plotting
data = pd.DataFrame(X, columns=["X1", "X2"])
data['cluster'] = y
sns.scatterplot(data,
                x = "X1",
                y = "X2",
                hue = "cluster",
                marker=".",
                s=40,
                lw=0,
                alpha=0.7,
                palette='tab10')
plt.xlabel("Feature X1")
plt.ylabel("Feature X2")
plt.title("Scatter plot of Blob Data")
# Create a legend
plt.legend(title="Blob Identity", loc="upper left")
```

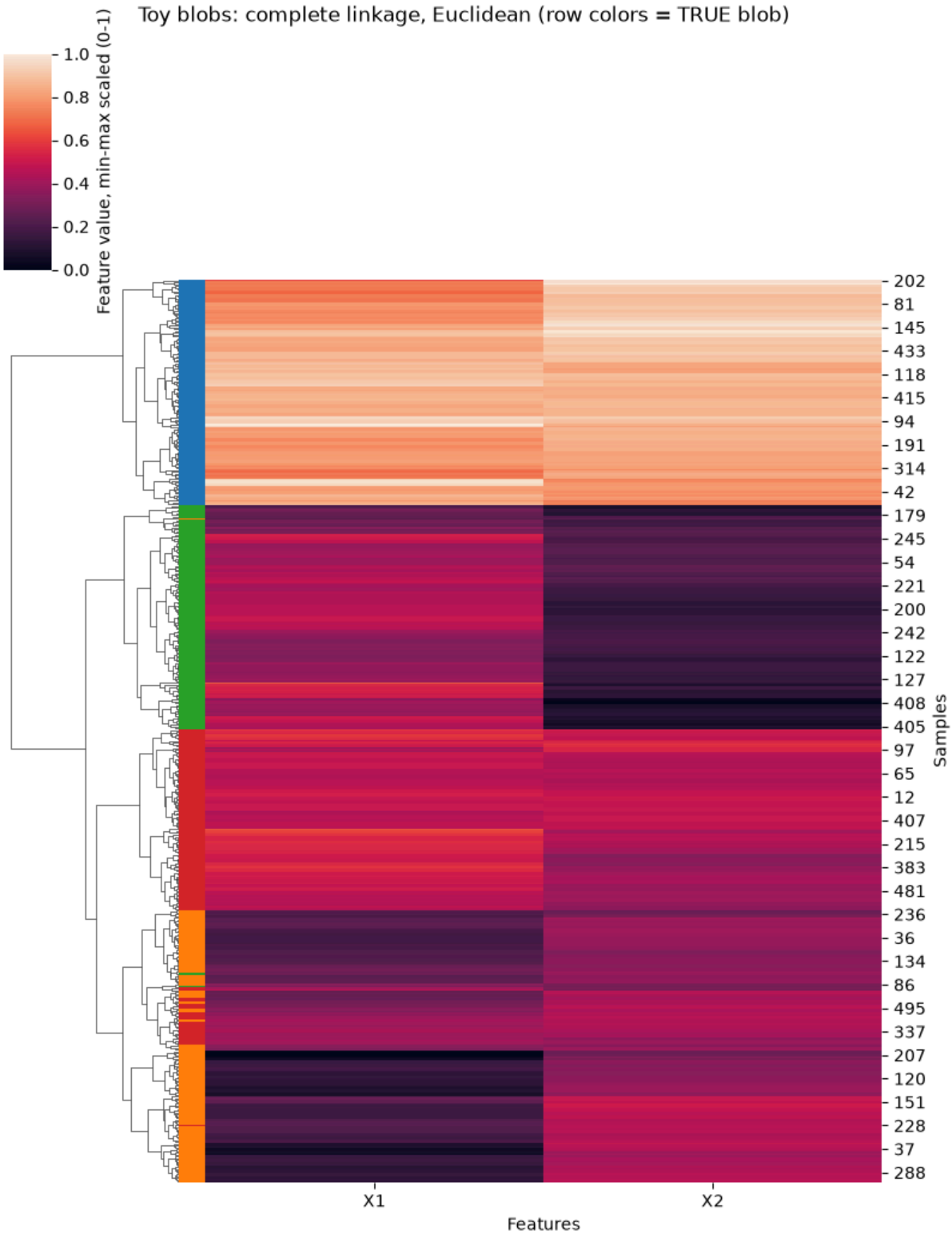
Out[ ]:
```
<matplotlib.legend.Legend at 0x116e0f250>
```



❓ 3.1 Write additional code to do hierarchical clustering on these data using an appropriate linkage type and print a heatmap. As before, title and label your heatmap and accompanying colorbar. Also mark the *true* blob identity `row_colors` on the heatmap. *Do not use the hierarchical clustering to create cluster assignments yet. Visualize the true blob identity, an external label, on this heatmap.*

In [ ]:
```
## YOUR CODE BELOW
# create hierarchical clustering heatmap with 2 features
XFt = data[["X1", "X2"]]
Z = fastcluster.linkage(XFt.values, method="complete", metric="euclidean")
palette = sns.color_palette("tab10", 4)
true_colors = [palette[i] for i in y]

g = sns.clustermap(
    XFt, row_linkage=Z, col_cluster=False,
    standard_scale=1, row_colors=true_colors, figsize=(8, 10),
    cbar_kws={"label": "Feature value, min-max scaled (0-1)"},
)
g.ax_heatmap.set_xlabel("Features")
g.ax_heatmap.set_ylabel("Samples")
g.fig.suptitle("Toy blobs: complete linkage, Euclidean", y=1.05)
plt.show()
print (XFt.shape)
print (X.shape)
# color by true blob identity
```




```
(500, 2)
(500, 2)
```

❓ 3.2 Cut the dendrogram at using a distance threshold and `fcluster`. Play with the distance threshold until you get 4 clusters. Create 3 plots:

1. A heatmap of the clustered features with the `row_colors` label reflecting the clusters chosen by `fcluster`. (This is what you made above, but with different `row_colors`.)
2. A silhouette plot for these chosen clusters from the complete linkage hierarchical clustering (hint: you might want to look ahead at the provided code for the next problem). *>Note that the code for the next problem is doing a lot more than just making a silhouette plot! Read the code and understand why it's using two for loops....you shouldn't need 2 for loops for this plot.*
3. A scatterplot of the data, colored by the hierarchical clustering cluster identity (not the true blob assignment)

In [ ]:
```
from sklearn.metrics import silhouette_samples, silhouette_score
from sklearn.cluster import KMeans
import matplotlib.cm as cm
```

```python
## YOUR CODE HERE
# Claude was used to debug
Z = fastcluster.linkage(XFt.values, method="complete", metric="euclidean")

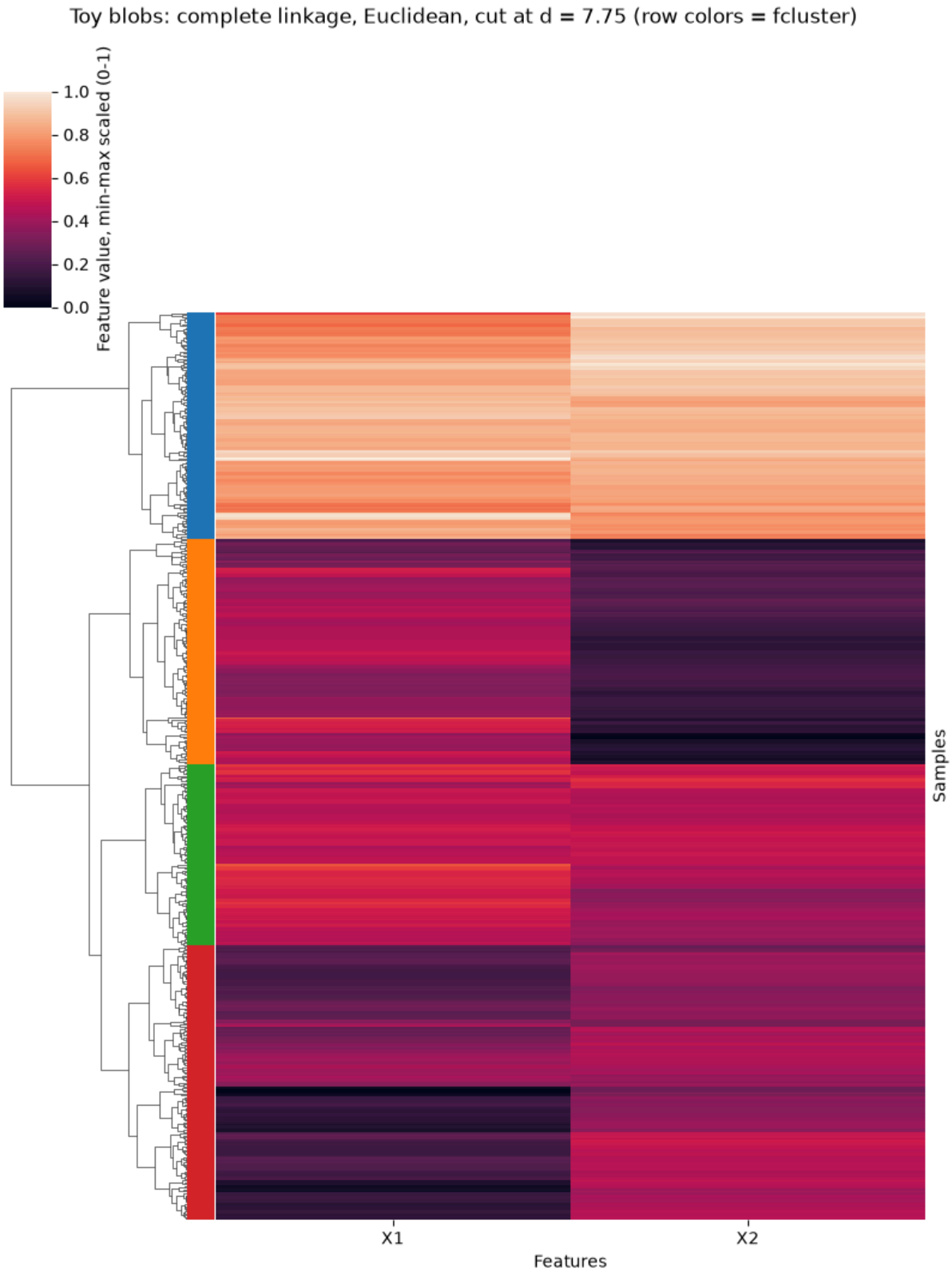
distance = Z[:, 2]
threshold, clusters = None, None
for t in (distance[:-1] + distance[1:]) / 2:
    c = fcluster(Z, t=t, criterion="distance")
    if len(np.unique(c)) == 4:
        threshold, clusters = t, c
        break
print("Distance threshold:", threshold, "| number of clusters:", len(np.unique(clusters)))

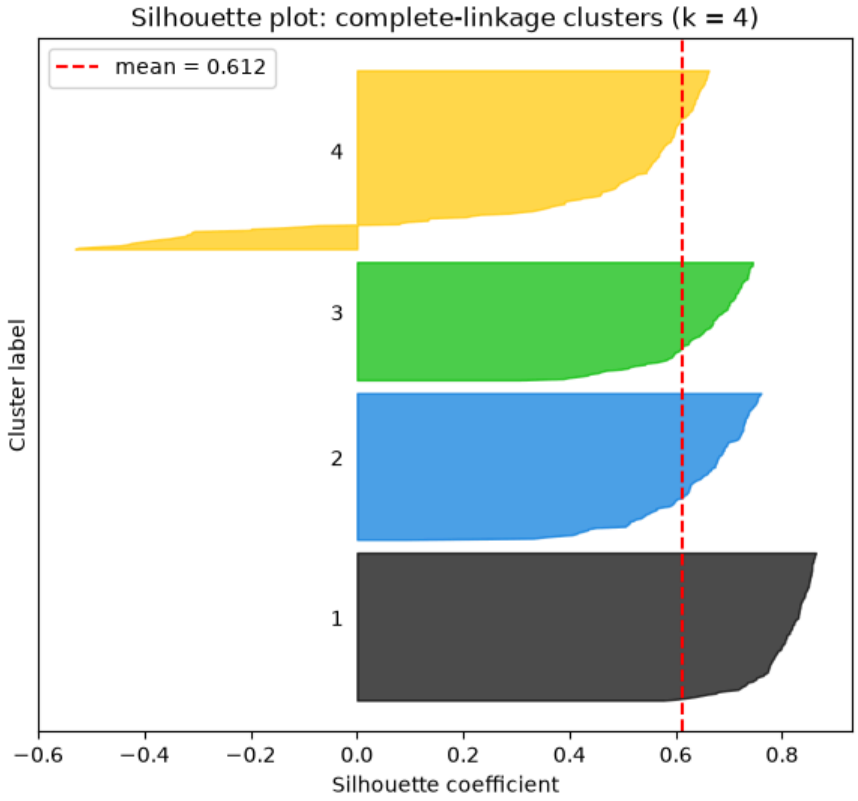
palette = sns.color_palette("tab10", 4)
row_colors = [palette[c - 1] for c in clusters]
g = sns.clustermap(
    XFt,
    row_linkage=Z,
    col_cluster=False,
    standard_scale=1,
    row_colors=row_colors,
    yticklabels=False,
    figsize=(8, 10),
    cbar_kws={"label": "Feature value, min-max scaled (0-1)"},
)
g.ax_heatmap.set_xlabel("Features")
g.ax_heatmap.set_ylabel("Samples")
g.fig.suptitle(f"Toy blobs: complete linkage, Euclidean, cut at d = {threshold:.2f} (row colors = fcluster)", y=1.05)
plt.show()

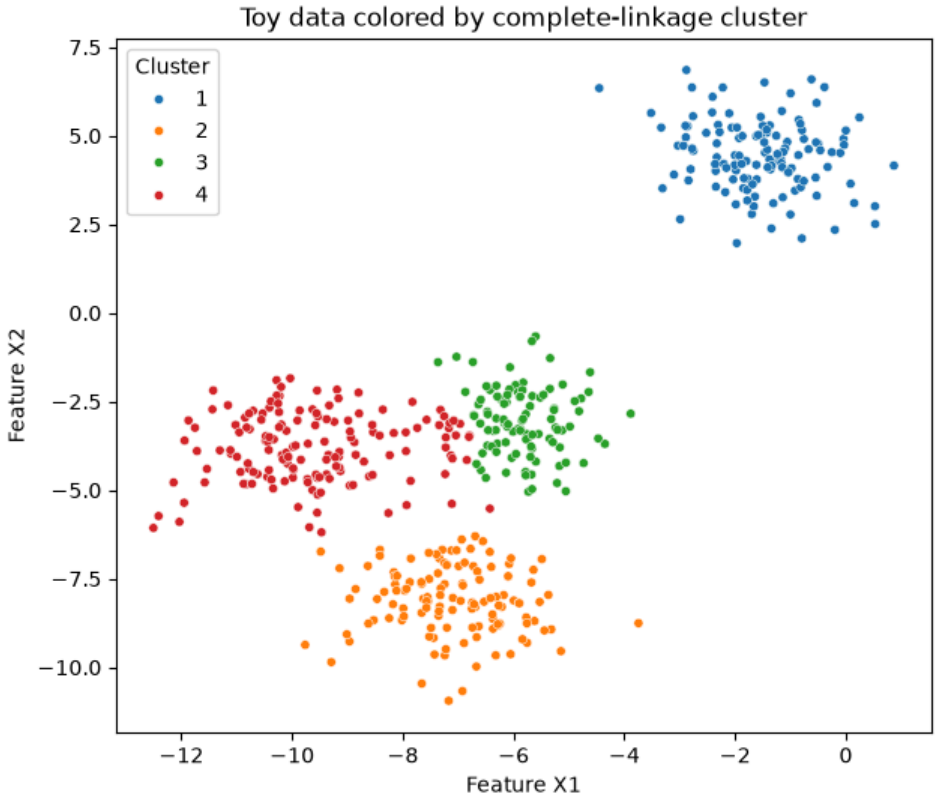
X_toy = XFt.to_numpy(dtype=float)
sil_avg = silhouette_score(X_toy, clusters)
sil_vals = silhouette_samples(X_toy, clusters)
fig, ax = plt.subplots(figsize=(7, 6))
y_lower = 10
for i, c in enumerate(np.unique(clusters)):
    vals = np.sort(sil_vals[clusters == c])
    y_upper = y_lower + len(vals)
    ax.fill_betweenx(np.arange(y_lower, y_upper), 0, vals,
                     color=cm.nipy_spectral(i / 4),
                     alpha=0.7)
    ax.text(-0.05, y_lower + len(vals) / 2, str(c))
    y_lower = y_upper + 10
ax.axvline(sil_avg, color="red", linestyle="--", label=f"mean = {sil_avg:.3f}")
ax.set_xlabel("Silhouette coefficient")
ax.set_ylabel("Cluster label")
ax.set_yticks([])
ax.set_title("Silhouette plot: complete-linkage clusters (k = 4)")
ax.legend()
plt.show()

plt.figure(figsize=(7, 6))
sns.scatterplot(x=X_toy[:, 0], y=X_toy[:, 1], hue=clusters, palette="tab10", s=20)
plt.xlabel("Feature X1")
plt.ylabel("Feature X2")
plt.title("Toy data colored by complete-linkage cluster")
plt.legend(title="Cluster")
plt.show()
```

```
Distance threshold: 7.745300706001446 | number of clusters: 4
```


Toy blobs: complete linkage, Euclidean, cut at d = 7.75 (row colors = fcluster)
Feature value, min-max scaled (0-1)
1.0
0.8
0.6
0.4
0.2
0.0
Samples
X1
X2
Features





❓ 3.3 Run the code below to cluster these data using k-means with k=2,3,4, 5, and 6 (use a for loop!). Inspect the resulting plots. Which value of k best suits these data? Is that the value you'd expect? Using what you know about silhouette scores, why is this the case?

In [22]:
```
import matplotlib.cm as cm
## YOUR CODE HERE
range_n_clusters = [2, 3, 4, 5, 6]


for n_clusters in range_n_clusters:
    # Create a subplot with 1 row and 2 columns
    fig, (ax1, ax2) = plt.subplots(1, 2)
    fig.set_size_inches(18, 7)

    # The 1st subplot is the silhouette plot
    # The silhouette coefficient can range from -1, 1 but in this example all
```

```
    # lie within [-0.1, 1]
    ax1.set_xlim([-0.1, 1])
    # The (n_clusters+1)*10 is for inserting blank space between silhouette
    # plots of individual clusters, to demarcate them clearly.
    ax1.set_ylim([0, len(X) + (n_clusters + 1) * 10])

    # Initialize the clusterer with n_clusters value and a random generator
    # seed of 10 for reproducibility.
    clusterer = KMeans(n_clusters=n_clusters, random_state=10)
    cluster_labels = clusterer.fit_predict(X)

    # The silhouette_score gives the average value for all the samples.
    # This gives a perspective into the density and separation of the formed
    # clusters
    silhouette_avg = silhouette_score(X, cluster_labels)
    print(
        "For n_clusters =",
        n_clusters,
        "The average silhouette_score is :",
        silhouette_avg,
    )

    # Compute the silhouette scores for each sample
    sample_silhouette_values = silhouette_samples(X, cluster_labels)

    y_lower = 10
    for i in range(n_clusters):
        # Aggregate the silhouette scores for samples belonging to
        # cluster i, and sort them
        ith_cluster_silhouette_values = sample_silhouette_values[cluster_labels == i]

        ith_cluster_silhouette_values.sort()

        size_cluster_i = ith_cluster_silhouette_values.shape[0]
        y_upper = y_lower + size_cluster_i

        color = cm.nipy_spectral(float(i) / n_clusters)
        ax1.fill_betweenx(
            np.arange(y_lower, y_upper),
            0,
            ith_cluster_silhouette_values,
            facecolor=color,
            edgecolor=color,
            alpha=0.7,
        )

        # Label the silhouette plots with their cluster numbers at the middle
        ax1.text(-0.05, y_lower + 0.5 * size_cluster_i, str(i))

        # Compute the new y_lower for next plot
        y_lower = y_upper + 10  # 10 for the 0 samples

    ax1.set_title("The silhouette plot for the various clusters.")
    ax1.set_xlabel("The silhouette coefficient values")
    ax1.set_ylabel("Cluster label")

    # The vertical line for average silhouette score of all the values
    ax1.axvline(x=silhouette_avg, color="red", linestyle="--")

    ax1.set_yticks([])  # Clear the yaxis labels / ticks
    ax1.set_xticks([-0.1, 0, 0.2, 0.4, 0.6, 0.8, 1])

    # 2nd Plot showing the actual clusters formed
    colors = cm.nipy_spectral(cluster_labels.astype(float) / n_clusters)
    ax2.scatter(
        X[:, 0], X[:, 1], marker=".", s=30, lw=0, alpha=0.7, c=colors, edgecolor="k"
    )

    # Labeling the clusters
    centers = clusterer.cluster_centers_
    # Draw white circles at cluster centers
    ax2.scatter(
        centers[:, 0],
        centers[:, 1],
        marker="o",
        c="white",
        alpha=1,
        s=200,
        edgecolor="k",
    )
```

```
    for i, c in enumerate(centers):
        ax2.scatter(c[0], c[1], marker="$%d$" % i, alpha=1, s=50, edgecolor="k")

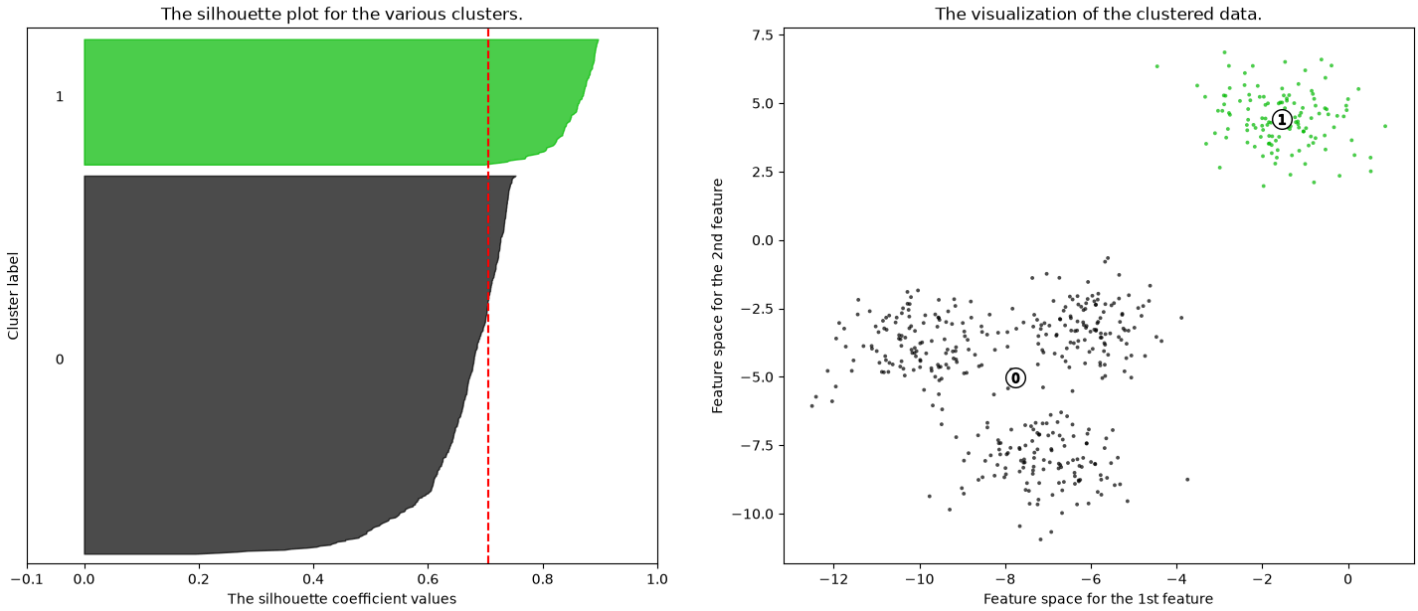
    ax2.set_title("The visualization of the clustered data.")
    ax2.set_xlabel("Feature space for the 1st feature")
    ax2.set_ylabel("Feature space for the 2nd feature")

    plt.suptitle(
        "Silhouette analysis for KMeans clustering on sample data with n_clusters = %d"
        % n_clusters,
        fontsize=14,
        fontweight="bold",
    )

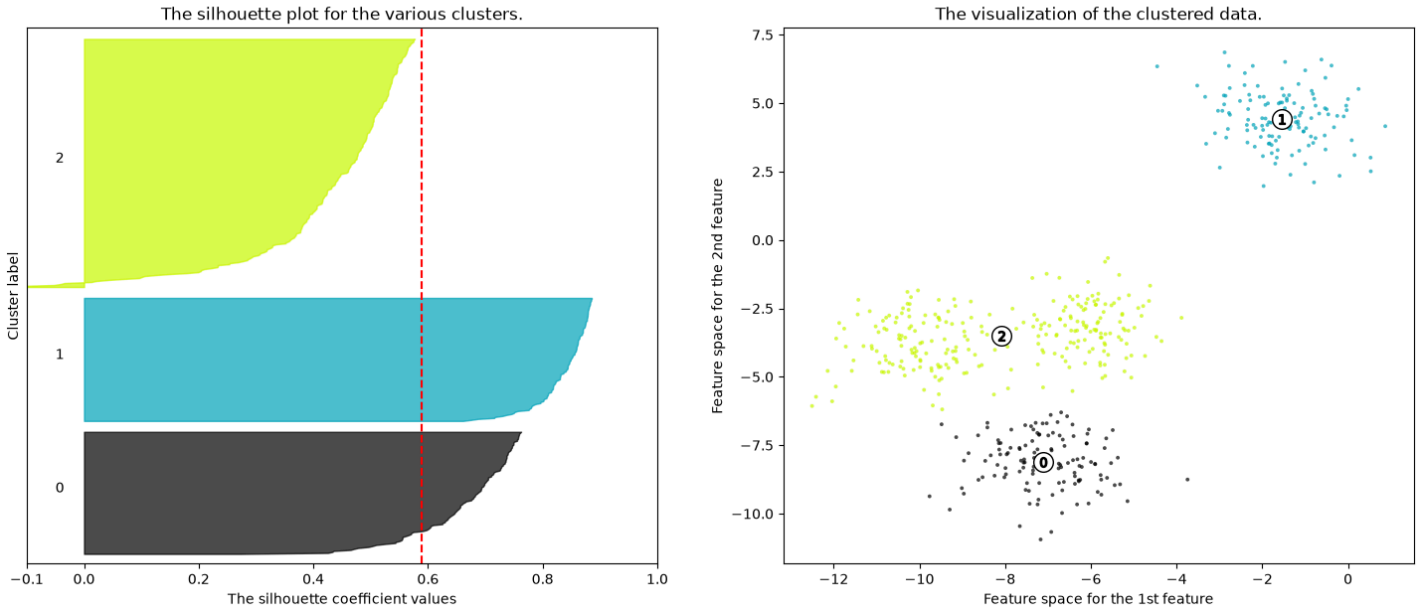
plt.show()
```

```
For n_clusters = 2 The average silhouette_score is : 0.7049787496083262
For n_clusters = 3 The average silhouette_score is : 0.5882004012129721
For n_clusters = 4 The average silhouette_score is : 0.6505186632729437
For n_clusters = 5 The average silhouette_score is : 0.561464362648773
For n_clusters = 6 The average silhouette_score is : 0.4857596147013469
```

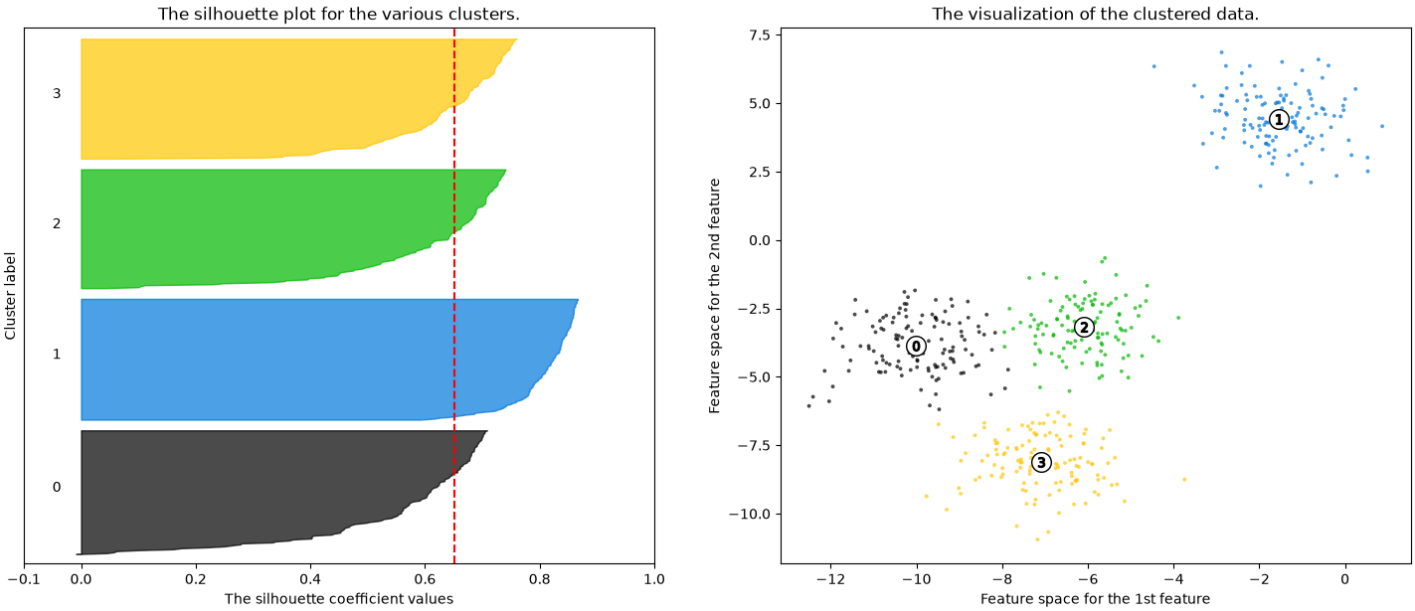
Silhouette analysis for KMeans clustering on sample data with n_clusters = 2



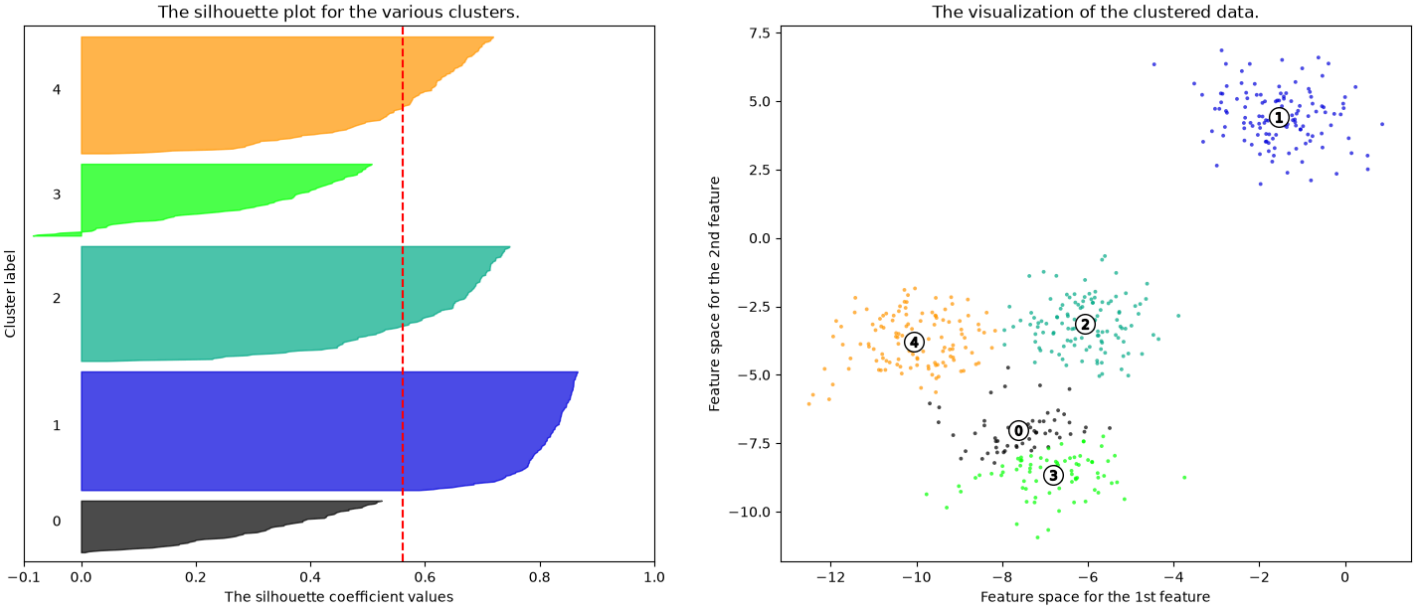
Silhouette analysis for KMeans clustering on sample data with n_clusters = 3



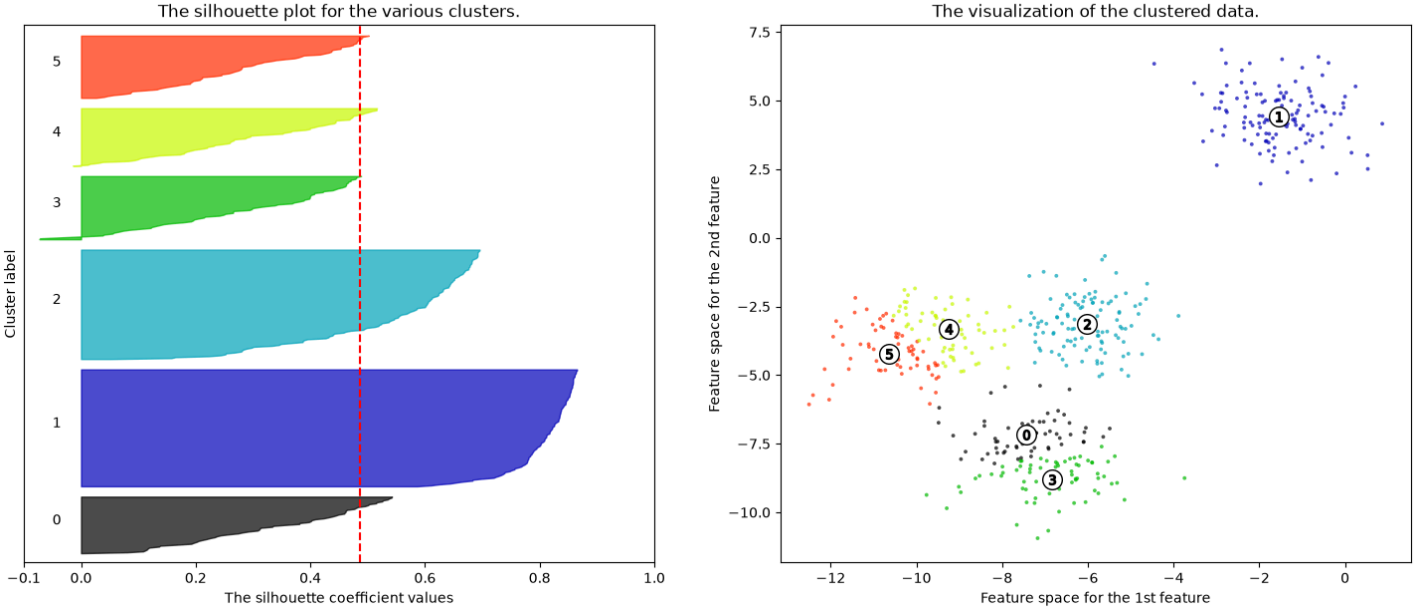
Silhouette analysis for KMeans clustering on sample data with n_clusters = 4



Silhouette analysis for KMeans clustering on sample data with n_clusters = 5



Silhouette analysis for KMeans clustering on sample data with n_clusters = 6



✅ Answer:

- k = 2 has the highest mean silhouette (0.705)
- Silhouette rewards a point being far from the nearest other cluster. The top-right cluster is far from everything, while the three bottom-left clusters touch each other
- Lumping those three clusters together gives every point a huge distance to the other cluster, so the score is high even though the grouping is wrong
- k = 4 is a close second (0.651), and its clusters match the real structure

❓ 3.4 We generated 4 clusters with k-means clustering and 4 clusters with hierarchical clusters from complete linkage. Are these the same clusters? Why or why not? (Consider: Should we expect these methods to match? What is each respective algorithm trying to optimize?)

✅ Answer: Although both methods recover the top-right and bottom blobs, they produce different clusters because complete linkage incorrectly merges part of the middle blob into the left blob, resulting in negative silhouette values for cluster 4. We should not expect them to match because they optimize entirely different objectives. Hierarchical clustering is a method that makes irreversible pairwise merges, whereas K-means iteratively reassigns points to minimize squared distances to cluster centers, making it better suited for round clusters.

⭐Section 4 - Bonus: Trying k-means on the real data

⭐ 4.1 The paper that compiled these T-ALL data identified 15 clusters. Before trying to run k-means on these data, do you think it will be able to identify 15 clusters well? Justify your answer using the hierarchical clustering from section 2 as evidence.

🍓 Answer: Probably not because hierarchical clustering cut at 15 produced one cluster of 1122 samples (1305 after scaling) plus singletons and tiny clusters. In Euclidean space the data doesn't separate into 15 similar-sized groups and that's exactly what k-means assumes. The paper's 15 subtypes probably rely on different features or preprocessing, so this doesn't mean the subtypes aren't real.

⭐ 4.2 Cluster these data using k-means, with k=15. Print a silhouette plot to evaluate the clustering. (Before printing your final plot, try several values of k. Do any of them look good?)

In [ ]:
```
## YOUR CODE HERE
# Claude was used to debug
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_samples, silhouette_score
import matplotlib.cm as cm

X_genes = dfTrans.to_numpy(dtype=float)
X_pca = PCA(n_components=2).fit_transform(X_genes)

range_n_clusters = [2, 3, 4, 6, 8, 10, 15]
scores = {}

for n_clusters in range_n_clusters:
    km = KMeans(n_clusters=n_clusters, random_state=10, n_init=10)
    labels = km.fit_predict(X_genes)
    sil_avg = silhouette_score(X_genes, labels)
    sil_vals = silhouette_samples(X_genes, labels)
    scores[n_clusters] = sil_avg
    print(f"k={n_clusters:2d}  mean silhouette={sil_avg:.3f}  sizes={sorted(np.bincount(labels), reverse=True)}")

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 7))
    y_lower = 10
    for i in range(n_clusters):
        vals = np.sort(sil_vals[labels == i])
        y_upper = y_lower + len(vals)
        color = cm.nipy_spectral(i / n_clusters)
        ax1.fill_betweenx(np.arange(y_lower, y_upper), 0, vals,
                          facecolor=color, edgecolor=color, alpha=0.7)
        ax1.text(-0.08, y_lower + len(vals) / 2, str(i))
        y_lower = y_upper + 10
    ax1.axvline(sil_avg, color="red", linestyle="--", label=f"mean = {sil_avg:.3f}")
    ax1.set_xlim([-0.2, 1])
    ax1.set_yticks([])
    ax1.set_xlabel("Silhouette coefficient")
    ax1.set_ylabel("Cluster label")
    ax1.set_title("Silhouette plot")
    ax1.legend()

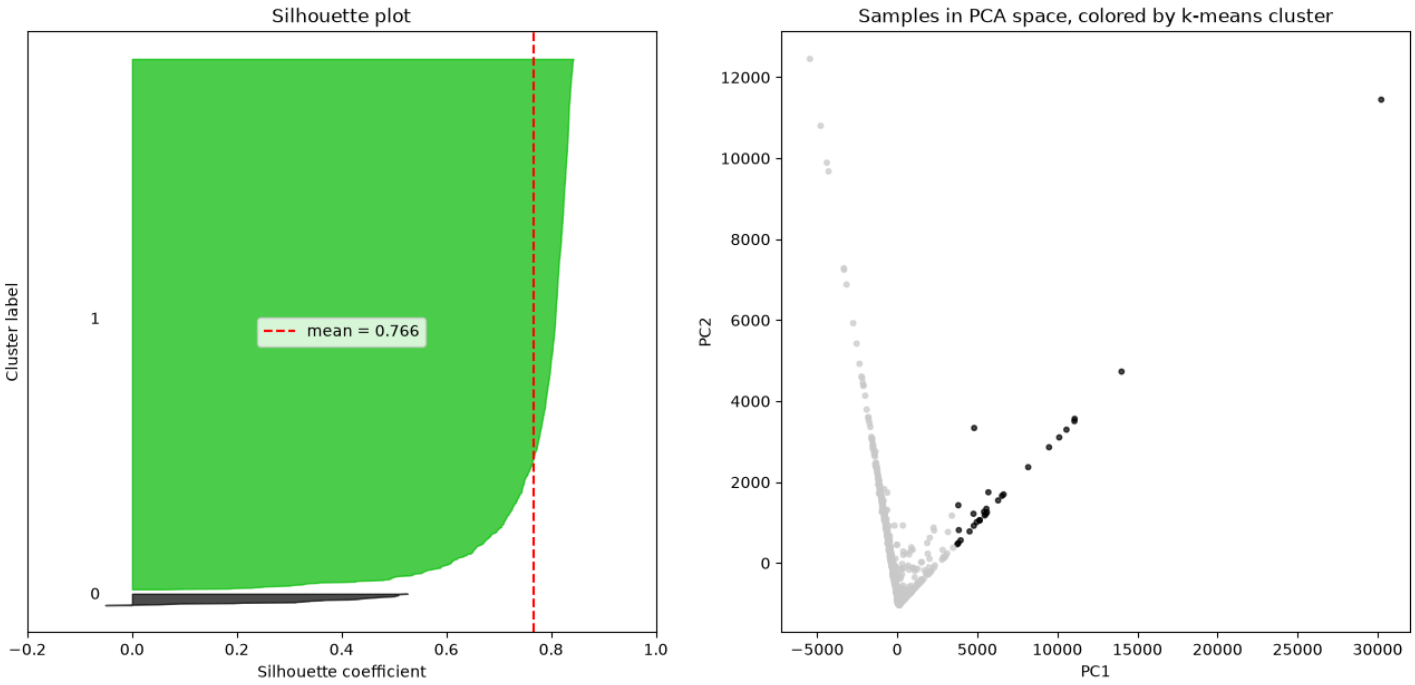
    ax2.scatter(X_pca[:, 0], X_pca[:, 1], c=labels, cmap="nipy_spectral", s=10, alpha=0.7)
    ax2.set_xlabel("PC1")
    ax2.set_ylabel("PC2")
    ax2.set_title("Samples in PCA space, colored by k-means cluster")

    fig.suptitle(f"k-means on T-ALL expression (raw TPM, Euclidean), k = {n_clusters}",
```

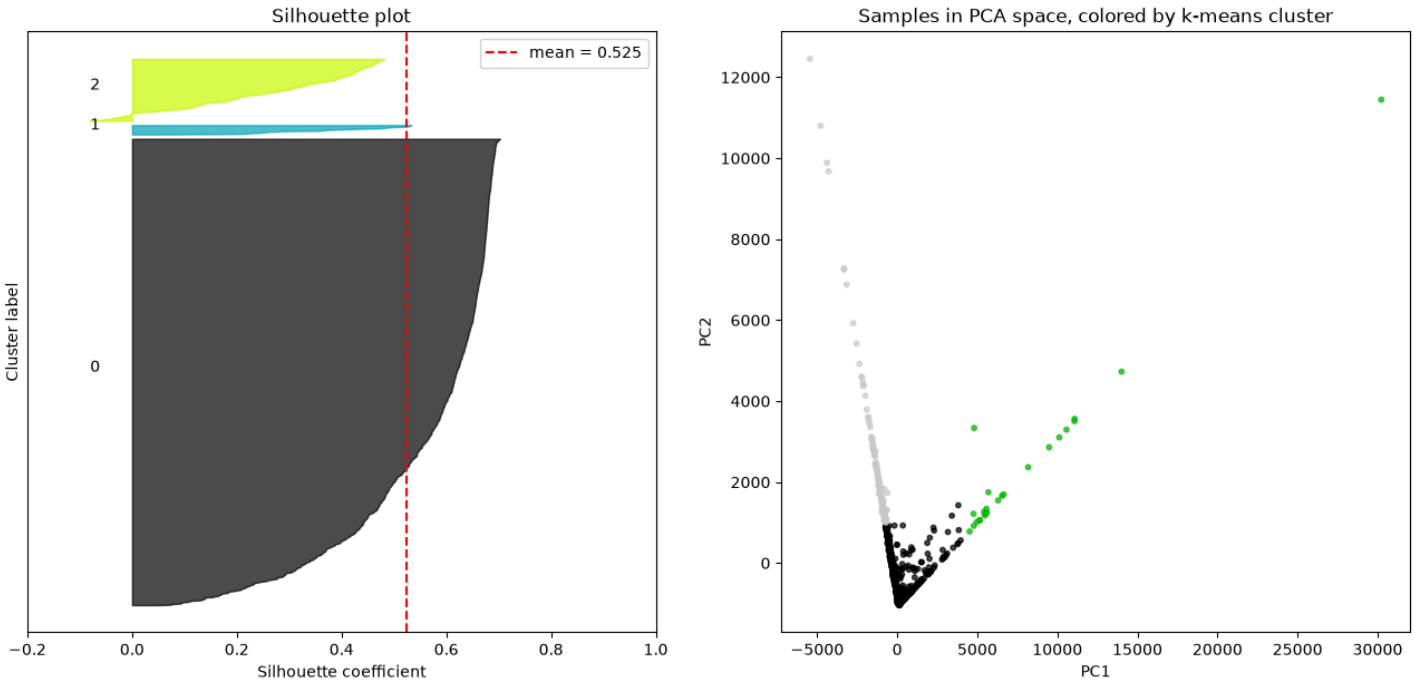
```
                fontsize=14, fontweight="bold")
    plt.show()

print(scores)
```

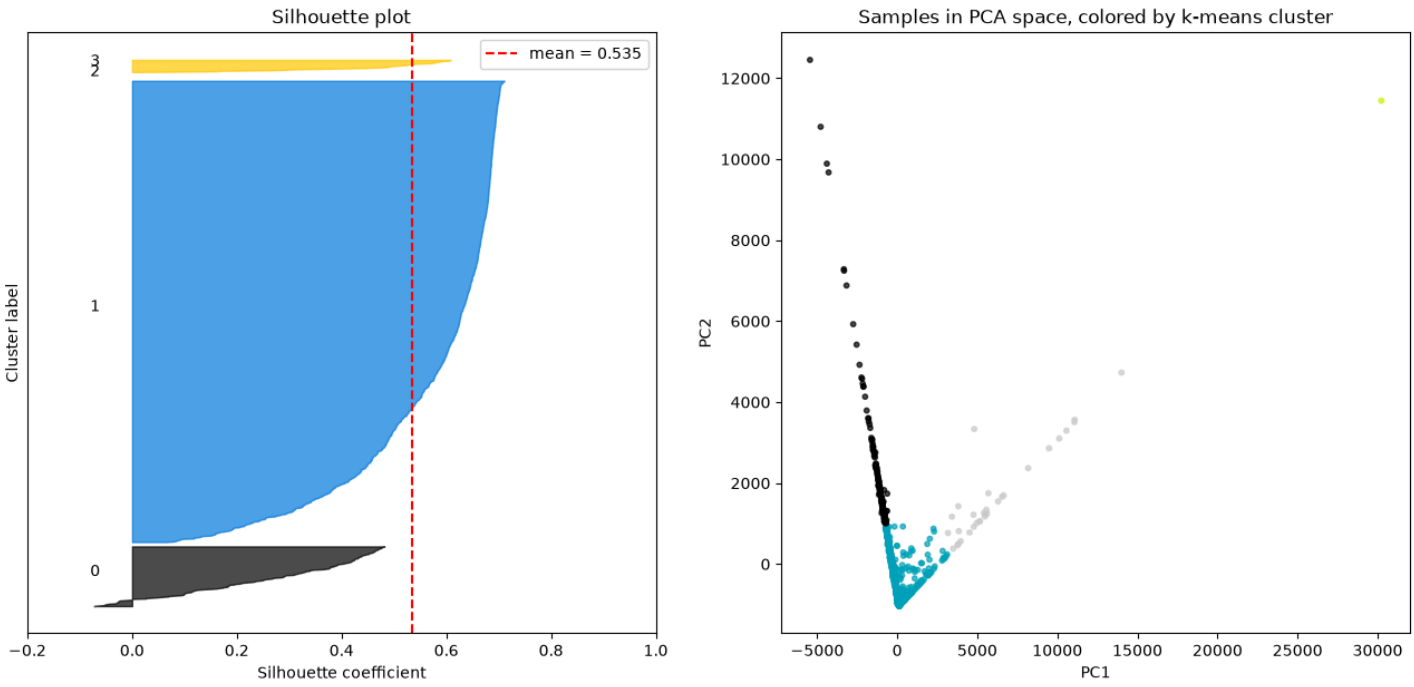
```
k= 2  mean silhouette=0.766  sizes=[np.int64(1306), np.int64(29)]
```



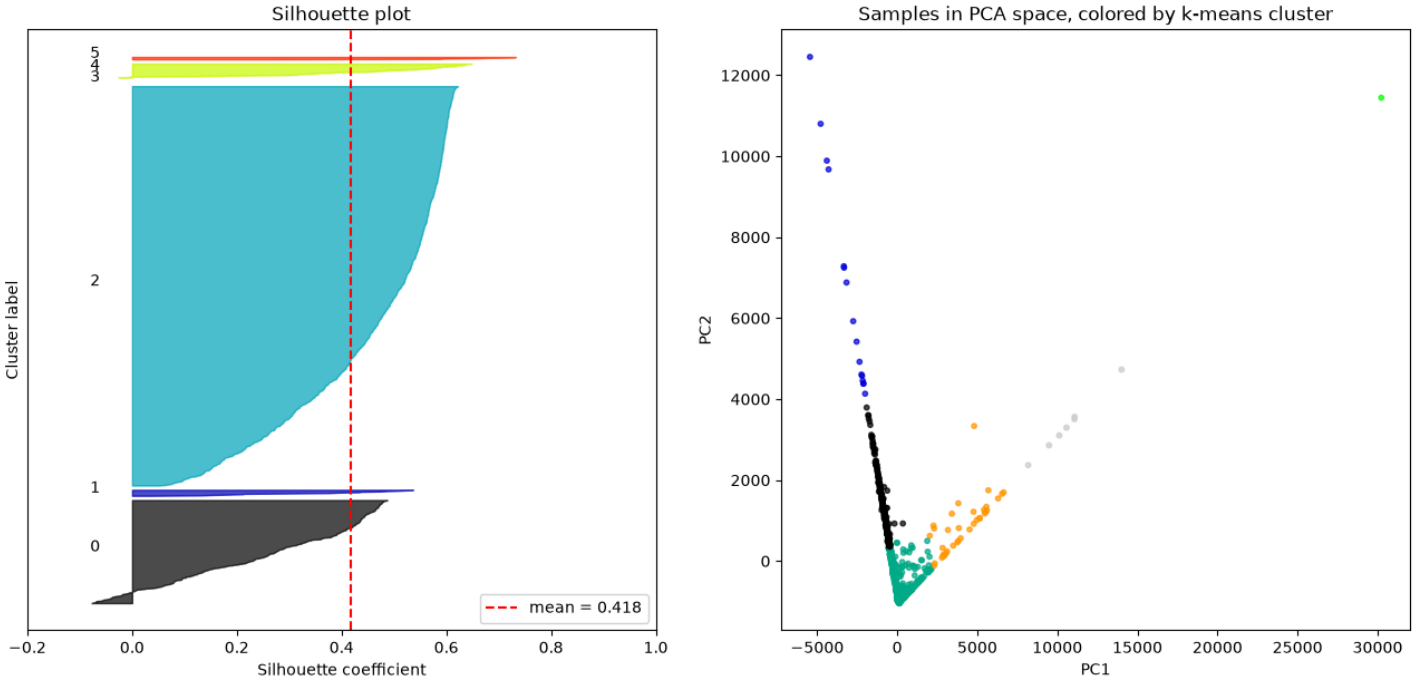

```
k= 3  mean silhouette=0.525  sizes=[np.int64(1157), np.int64(154), np.int64(24)]
```



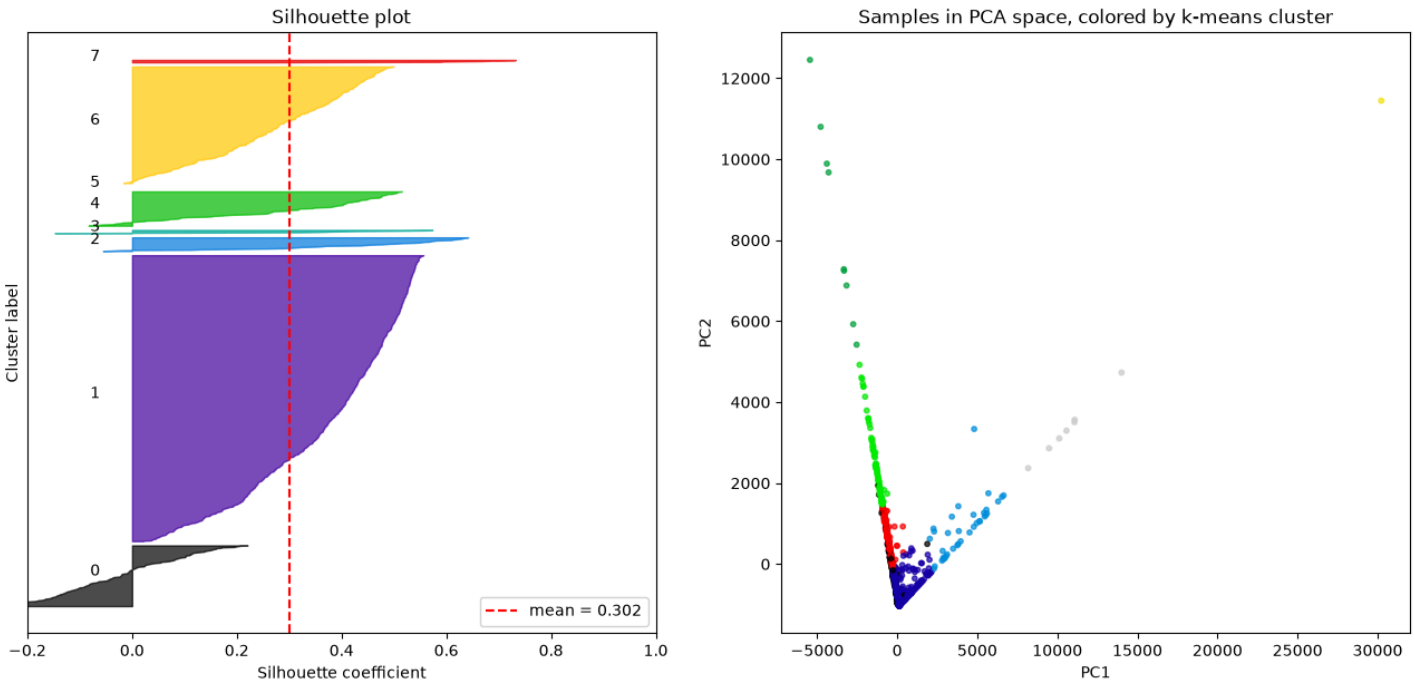

```
k= 4  mean silhouette=0.535  sizes=[np.int64(1153), np.int64(150), np.int64(31), np.int64(1)]
```



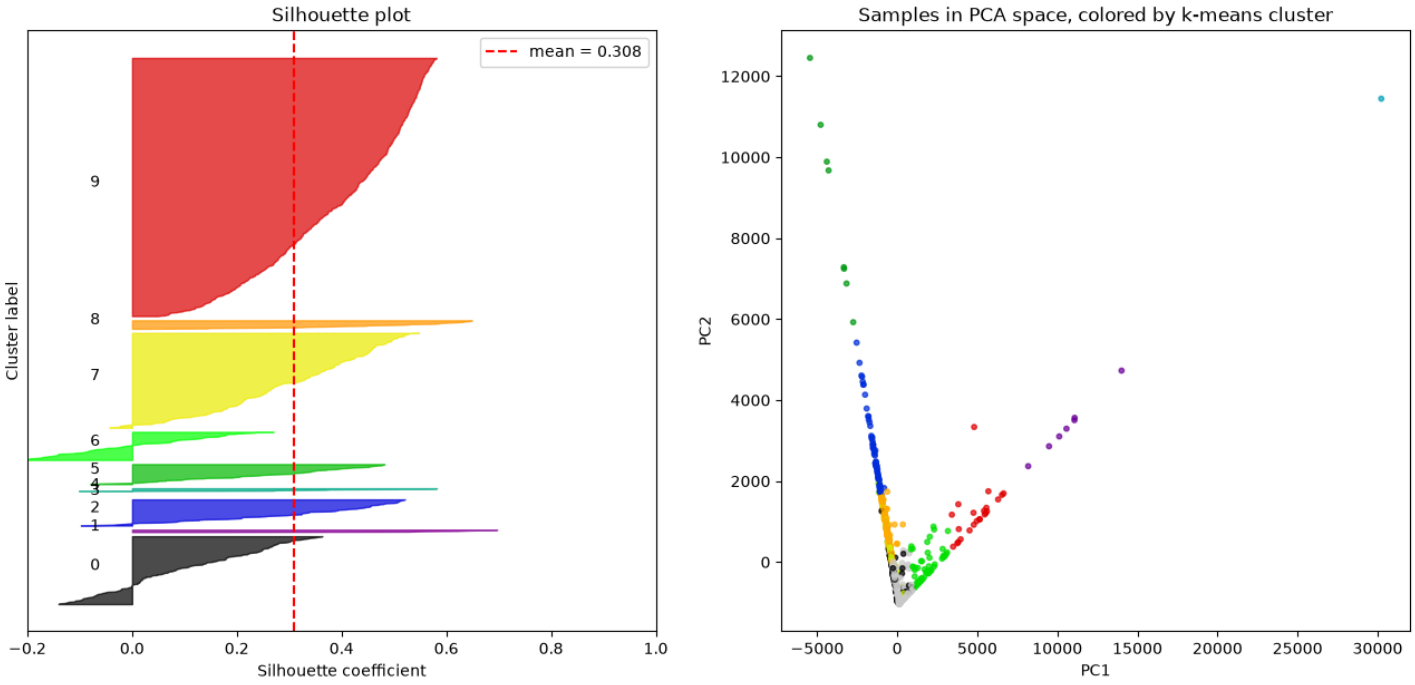
k= 6 mean silhouette=0.418 sizes=[np.int64(1013), np.int64(262), np.int64(36), np.int64(16), np.int64(7), np.int64(1)]



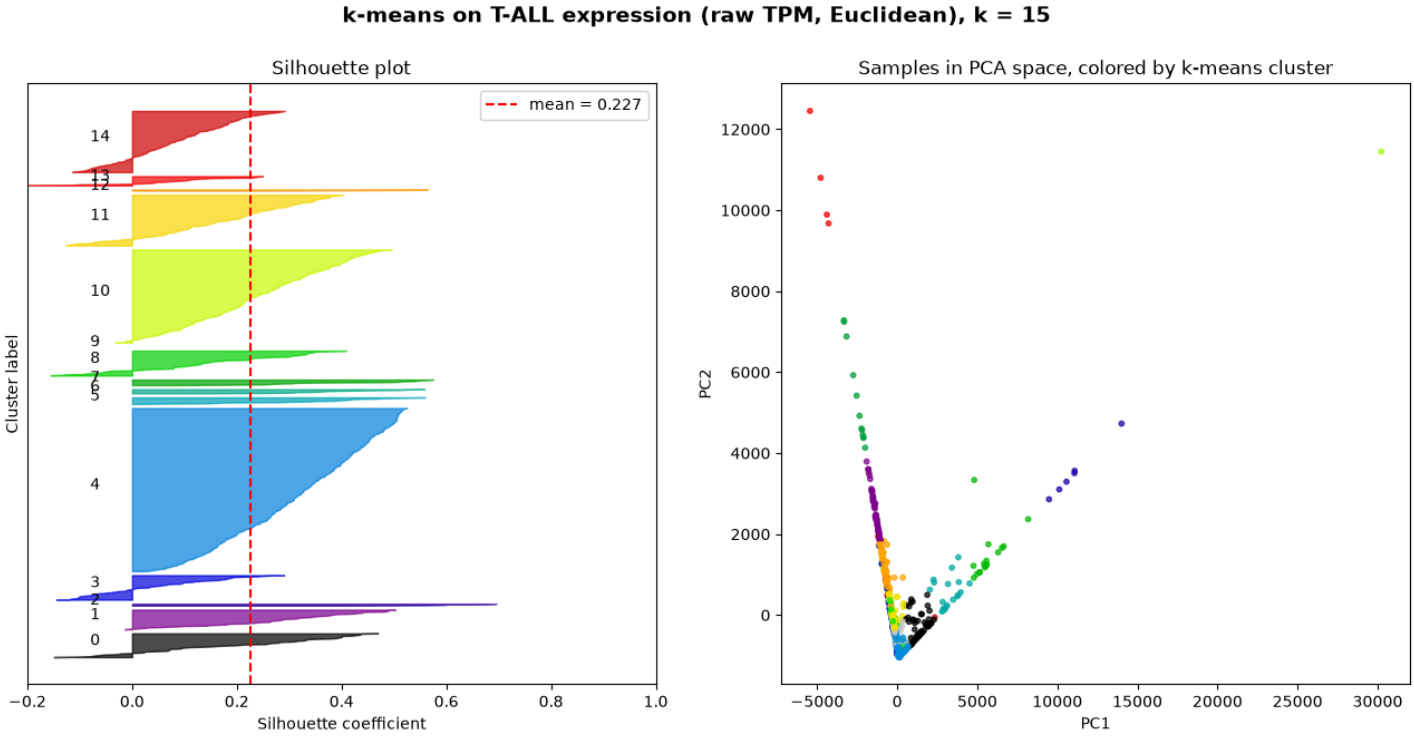
k= 8 mean silhouette=0.302 sizes=[np.int64(736), np.int64(300), np.int64(157), np.int64(89), np.int64(36), np.int64(9), np.int64(7), np.int64(1)]



k=10 mean silhouette=0.308 sizes=[np.int64(674), np.int64(248), np.int64(178), np.int64(74), np.int64(69), np.int64(53), np.int64(23), np.int64(8), np.int64(7), np.int64(1)]



k=15 mean silhouette=0.227 sizes=[np.int64(441), np.int64(252), np.int64(166), np.int64(138), np.int64(68), np.int64(67), np.int64(65), np.int64(54), np.int64(26), np.int64(19), np.int64(16), np.int64(12), np.int64(6), np.int64(4), np.int64(1)]



{2: 0.766123998456289, 3: 0.5251223235643522, 4: 0.5348790058887717, 6: 0.41787197705218704, 8: 0.3015677154517475, 10: 0.3080533114990785, 15: 0.22659264283366307}

⭐ 4.3 What are some reasons k-means likely has trouble on these data?

🍓 Answer: Using raw, unscaled TPM with many uninformative genes and heavy-tailed outliers stretching up to 30,000 causes a few high-expression genes to dominate distances and skew cluster centers. Because K-means assumes round clusters of similar size and spread, it poorly fits the unequal, V-shaped real groups shown in the PCA plot. Furthermore, the algorithm requires you to choose k in advance and is sensitive to random starting centers.

Midsemester Feedback

Required, 1 pt each for completion.

❓ What is one thing you think these labs have taught well? *Could be a general response, or a more specific comment about a specific lab.*

✅ Answer: I think the labs are great to help me visualize the methods that went over my head in lecture.

❓ What is one thing you'd like to be different about these labs? *Could be a general response, or a more specific comment about a specific lab.*

✅ Answer: More bonus questions. (I am glad it was brought back for this lab...I've missed it over the past few 😢)